

HANDS-ON 7 [Intermediate]

FastAPI — Dependency Injection, CRUD & OpenAPI Documentation

Topics Covered:

FastAPI Dependency Injection✓

CRUD Operations✓

Response Models & Status Codes✓

Background Tasks✓

OpenAPI Customisation✓

Error Handling with HTTPException✓

You will deepen your FastAPI skills by completing the full CRUD implementation, customising the OpenAPI documentation, handling errors properly, and using FastAPI's background tasks feature.

Task 1: Complete CRUD with Proper HTTP Conventions

Goal: Implement all CRUD endpoints following REST conventions for status codes and response models.

Steps:

68.

Complete PUT /api/courses/{id} (update) and DELETE /api/courses/{id} (delete). Use response_model=CourseResponse on GET and POST endpoints to document and validate the response shape.

69.

Use status_code=status.HTTP_201_CREATED on the POST endpoint and status_code=status.HTTP_204_NO_CONTENT on DELETE (no response body).

70.

Use `HTTPException(status_code=404, detail='Course not found')` when a course ID does not exist. FastAPI converts this to a JSON error response automatically.

71.

Add a `GET /api/courses/{id}/students/` endpoint that returns all students enrolled in the course, using a JOIN query.

72.

Implement all CRUD for Students and Enrollments following the same patterns.

Hint:

-

`response_model` tells FastAPI what the response should look like — it filters out any extra fields from the ORM model that should not be exposed.

-

HTTP 204 No Content is the correct status for successful DELETE — return nothing (just status code), not an empty JSON object.

Expected Outcome: DELETE returns 204 with no body. POST returns 201 with the created resource. Invalid IDs return 404 with a JSON detail message.

Task 2: Background Tasks and OpenAPI Customisation

Goal: Use FastAPI Background Tasks and customise the OpenAPI documentation.

Steps:

73.

Add a `BackgroundTasks` parameter to the `POST /api/enrollments/` endpoint. After creating an enrollment, add a background task that simulates sending a confirmation email: `def send_confirmation_email(student_email: str): print(f'Sending confirmation to {student_email}')`.

74.

Verify the endpoint returns immediately (201) without waiting for the background task — check the server console for the print output after the response.

75.

Customise the OpenAPI metadata in the `FastAPI()` constructor: add title, description, version, and contact information.

76.

Add tags to group related endpoints: `@app.get('/api/courses/', tags=['Courses'])`. Observe how the Swagger UI organises endpoints by tag.

77.

Add `response_description` and `summary` to the `@app.post` decorator for the create course endpoint. Check how this appears in the `/docs` UI.

Hint:

-

Background tasks run after the response is sent — useful for non-critical work like sending emails or logging, but not for operations the client needs to wait for.

Page | For Digital Nurture 5.0 Participants Only

Digital Nurture 5.0 | Python Backend Frameworks | Hands-On Exercise Book

- **OpenAPI tags and descriptions make your /docs page self-documenting — anyone can understand and test your API without reading the source code.**

Expected Outcome: POST /api/enrollments/ returns 201 immediately. Server console shows the email confirmation print after the response. /docs shows grouped, described endpoints.

OUTCOME:

Post api/student:

The screenshot displays a web browser window with multiple tabs open, including 'schemas.py', 'models.py', 'Course Management System - Swa...', 'config.py', 'database.py', and 'main.py'. The address bar shows the URL 'http://127.0.0.1:8000/docs#/Students/create_student_api_students__post'. The main content area is titled 'Responses' and shows the details for a 201 status code response. The 'Curl' section contains the following command:

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/students/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Hamza Farooque",
    "email": "hamza@example.com"
  }'
```

The 'Request URL' is 'http://127.0.0.1:8000/api/students/'. The 'Server response' section shows the response body as a JSON object:

```
{
  "name": "Hamza Farooque",
  "email": "hamza@example.com",
  "id": 1
}
```

The 'Response headers' section shows the following headers:

```
content-length: 60
content-type: application/json
date: Wed, 01 Jul 2026 08:37:41 GMT
server: uvicorn
```

The bottom of the page shows a 'Responses' table with columns 'Code' and 'Description', and a 'Links' section with a 'Go Live' button.

POST api/enrollments:

schemas.pymodels.pyCourse Management System - Swa... Xconfig.pydatabase.pymain.py

← → http://127.0.0.1:8000/docs#/Courses/create_course_api_courses_post

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/enrollments/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "student_id": 1,
    "course_id": 1
  }'
```

Request URL

http://127.0.0.1:8000/api/enrollments/

Server response

Code	Details
201	Response body <pre>{ "student_id": 1, "course_id": 1, "id": 1, "enrollment_date": "2026-07-01T08:48:04.301757" }</pre>  Download Response headers <pre>content-length: 84 content-type: application/json date: Wed, 01 Jul 2026 08:48:03 GMT server: uvicorn</pre>

Responses